Graph Representation Learning: From-Scratch and PyTorch Geometric Implementations

A progression of graph representation learning methods implemented for the M.Tech project, moving from unsupervised random-walk embeddings (DeepWalk, node2vec) to message-passing graph neural networks (GCN, GraphSAGE, GIN, SGC). Earlier methods are implemented entirely from first principles in NumPy and PyTorch for transparency; later methods are implemented with PyTorch Geometric (PyG) once the underlying mechanics have already been verified by hand, and are used to run a structured set of experiments probing GNN behavior (oversmoothing, oversquashing, permutation equivariance, hyperparameter sensitivity, and Weisfeiler-Lehman expressivity).

Motivation

Most public implementations of these methods rely on a library (`gensim.Word2Vec`, `torch_geometric.nn.GCNConv`) as a black box. This project re-derives and re-implements the mechanics described in each paper wherever the goal is to understand the method, so that the relationship between graph structure, propagation, and the resulting representations can be examined directly at each stage. Once a method's correctness has been established from scratch, later work uses the equivalent PyTorch Geometric layer directly, so that effort goes into designing and reading the experiments rather than re-deriving code that has already been verified.

Repository Structure

```
.
|-- DeepWalk_Node2Vec_Implementation.ipynb  DeepWalk and node2vec, implemented from scrat
|-- data/
|      `--  Homo_sapiens.mat                                          Protein-
protein interaction network (node2vec paper's benchmark dataset)
|-- results/
|  |-- comparison.png               2D PCA projection of DeepWalk vs. node2vec embeddings
|  |-- karate.png                  DeepWalk embeddings on the Zachary Karate Club graph
|  `-- output.png                  Precision@1 across five random seeds (robustness check
|-- GCN_and_GraphSAGE/
|  |-- GCN_Cora_Implementation.ipynb     GCN implemented from scratch and with PyG, side b
|  |-- GCN_GraphSAGE_PyG_Experiments.ipynb  GCN, GraphSAGE, GIN and SGC (PyG layers only) p
guide experiment suite
|  |-- data/                     Cora dataset cache, downloaded automatically by PyG on fi
|   `-- OUTPUT/
|       |-- output.png                        Training loss, from-scratch GCN
|       |-- output2.png                        Training loss, PyG GCN
|             |--  output3.png                                          From-
scratch vs. PyG GCN accuracy (train/val/test)
|     `-- output4.png               Test accuracy vs. GCN depth (oversmoothing)
|-- rough/                        Early exploratory notebooks and superseded drafts, kept
```

```
|-- requirements.txt
`-- README.md
```

Part 1 – DeepWalk and node2vec (from scratch)

An implementation of two foundational graph representation learning algorithms, DeepWalk (Perozzi et al., 2014) and node2vec (Grover and Leskovec, 2016), built entirely from first principles using NumPy and PyTorch. Every component, including random walk generation, skip-gram context construction, hierarchical softmax, negative sampling, and biased second-order walks, is implemented as an explicit, inspectable function rather than invoked from an existing graph-embedding library.

DeepWalk - Unbiased, first-order random walks over the graph's transition matrix - Context pair generation via a sliding window over each walk - Skip-gram objective optimized with Hierarchical Softmax, implemented as an explicit binary tree over the vocabulary of nodes, matching the method described in the original paper

node2vec - Biased, second-order random walks governed by the return parameter p and the in-out parameter q - The alpha_pq(t, x) transition bias, derived and implemented directly from the paper's formulation - Alias sampling (Walker's method) for O(1) sampling of the next walk step - Skip-gram objective optimized with Negative Sampling, as specified in the original paper

Both algorithms share the same skip-gram training loop; they differ only in how the training pairs are generated (unbiased vs. biased walks) and how the softmax is approximated (hierarchical softmax vs. negative sampling), consistent with each paper's respective design.

Experiments

1. **Toy graph baseline** — a fixed 6-node graph used to validate correctness of every function in isolation, including random walks, transition probabilities, and skip-gram training.
2. **node2vec parameter study** — walks and embeddings generated under three (p, q) settings: balanced (p=1, q=1), BFS-like (p=0.5, q=2), and DFS-like (p=2, q=0.5), evaluated for how each setting shapes the embedding space.
3. **Real-world validation on the Zachary Karate Club graph** (34 nodes, 78 edges) — the identical, unmodified pipeline applied to a real social network with a known community structure.
4. **Real-world validation on the Homo sapiens PPI network** (3,890 nodes, 38,292 edges, 50 GO-annotation labels) — the same dataset used in the original node2vec paper, evaluated with multi-label classification (macro/micro-F1) via a one-vs-rest logistic regression classifier, using a vectorized negative-sampling implementation for scalability.
5. **Multi-seed robustness check** — the full DeepWalk vs. node2vec comparison repeated across five random seeds, reporting mean and standard deviation of loss and precision@k rather than a single run.
6. **Structural equivalence correlation** — an evaluation metric independent of precision@k, measuring whether nodes of similar degree are placed close to-

gether in embedding space even when not directly connected.

7. **Hierarchical softmax and alias sampling, verified in isolation** — standalone demonstrations confirming that hierarchical softmax probabilities sum to one over the binary tree, and that alias sampling reproduces the correct biased transition distribution.

Evaluation Metrics

- **Precision@k** — for each node, the fraction of its k nearest neighbors in embedding space (by cosine similarity) that are true graph neighbors. Measures how well the embedding preserves local adjacency.
- **Structural equivalence correlation** — the correlation between embedding similarity and degree similarity across non-adjacent node pairs. Measures whether the embedding captures structural role rather than direct connectivity.
- **Macro/Micro-F1** (PPI dataset only) — standard multi-label classification metrics used in the original node2vec paper for evaluating embedding quality on a downstream task.

Part 2 – GCN and GraphSAGE

Graph convolutional networks (Kipf and Welling, 2017) and GraphSAGE (Hamilton, Ying and Leskovec, 2017), studied on the Cora citation network (Planetoid split: 2,708 nodes, 10,556 edges, 1,433 features, 7 classes, 140 training labels).

`GCN_Cora_Implementation.ipynb`

A GCN layer implemented twice on the same data and the same training loop: once by hand as `Â X W` with `Â = D̃^(-1/2)(A+I)D̃^(-1/2)` built explicitly, and once with `torch_geometric.nn.GCNConv`. The two are trained side by side so that the from-scratch implementation can be checked against the library implementation directly, then a depth sweep (1 to 15 layers, no residual connections, no normalization tricks) demonstrates oversmoothing empirically.

`GCN_GraphSAGE_PyG_Experiments.ipynb`

A second notebook built entirely on PyTorch Geometric layers — `GCNConv`, `SAGEConv`, `GINConv`, `SGConv` — with no hand-written message passing, that trains GCN and GraphSAGE on Cora and then works through the lab-guide experiment suite:

- **Oversmoothing** — propagation-only Dirichlet energy and cosine-similarity decay, a trained-model depth sweep, and the residual-connection / DropEdge fixes overlaid against the unfixed baseline.
- **Oversquashing** — Jacobian sensitivity of a 6-layer untrained GCN on barbell, ring, and complete graphs, plus a NeighborsMatch-style tree task showing train accuracy collapse as the required routing radius grows.
- **Permutation equivariance and invariance** — `GCN(PX, PAPᵀ) = P·GCN(X, A)` verified numerically, a sum-readout graph invariant confirmed, and the guaran-

tee deliberately broken by adding node index as a feature or replacing the GCN with an MLP on the flattened adjacency matrix.
- **Controlled hyperparameter study (OFAT)** — depth, hidden size, dropout, weight decay, and learning rate swept one at a time around the Kipf and Welling baseline, plus a GCN-vs-MLP comparison as the number of labels per class shrinks.
- **Weisfeiler-Lehman expressivity** — 1-WL color refinement implemented by hand, run against WL-hard and WL-easy graph pairs, and compared to untrained GCN, mean-aggregation GraphSAGE, and sum-aggregation GIN embeddings; followed by a degree-vs-triangle-count node regression showing an expressivity ceiling that no amount of training closes.
- **Bonus: SGC** — `SGConv` (propagation with no per-layer weights or nonlinearity) compared against the full GCN.

Output analysis (`GCN_and_GraphSAGE/OUTPUT/`)

The four saved plots are from `GCN_Cora_Implementation.ipynb`:

- **`output.png` / `output2.png`** — training loss for the from-scratch GCN and the PyG GCN over 200 epochs. Both curves fall from about 1.95 to roughly 0.35-0.40 with the same shape and the same plateaus, and the PyG version converges very slightly faster — evidence that the hand-written propagation matrix matches what `GCNConv` computes internally.
- **`output3.png`** — final train/val/test accuracy, from-scratch vs. PyG. The two implementations land within about one point of each other on every split (train ≈ 0.99, val ≈ 0.78-0.79, test ≈ 0.80-0.82), which is the direct correctness check for the from-scratch layer.
- **`output4.png`** — test accuracy vs. number of GCN layers, 1 to 15, no residual connections or normalization added. Accuracy peaks at 2-3 layers (≈ 0.80), collapses sharply by 4-8 layers to about 0.13-0.15, and stays low with minor noise beyond that — the oversmoothing effect predicted by the theory, reproduced empirically on real data before any of the fixes in `GCN_GraphSAGE_PyG_Experiments.ipynb` are applied.

Evaluation Metrics

- **Node classification accuracy** (train / val / test splits) — the standard Planetoid semi-supervised setting.
- **Dirichlet energy** — quantifies how spread apart node representations are across graph edges; used to measure oversmoothing directly rather than inferring it from accuracy alone.
- **Jacobian sensitivity** — ‖∂h_i/∂x_j‖, used to measure how much a distant node's input can still influence a target node's output, i.e. oversquashing.
- **1-WL distinguishability** — whether the Weisfeiler-Lehman color-refinement histogram differs between two graphs, used as the ceiling against which GNN expressivity is measured.

Requirements

See `requirements.txt`. Developed and tested with Python 3.10+.

```
pip install -r requirements.txt
```

`torch_geometric` is required for `GCN_and_GraphSAGE/`; it is not needed for the DeepWalk/node2vec notebook.

Usage

- `DeepWalk_Node2Vec_Implementation.ipynb` — open and run all cells in order. The toy graph and the Zachary Karate Club edge list are defined inline, and the PPI dataset is loaded from `data/Homo_sapiens.mat`.
- `GCN_and_GraphSAGE/GCN_Cora_Implementation.ipynb` and `GCN_and_GraphSAGE/GCN_GraphSAGE_` — open and run all cells in order. Cora is downloaded automatically by PyTorch Geometric into `GCN_and_GraphSAGE/data/` on first run.

References

- B. Perozzi, R. Al-Rfou, and S. Skiena. "DeepWalk: Online Learning of Social Representations." KDD 2014.
- A. Grover and J. Leskovec. "node2vec: Scalable Feature Learning for Networks." KDD 2016.
- W. W. Zachary. "An Information Flow Model for Conflict and Fission in Small Groups." Journal of Anthropological Research, 1977.
- T. N. Kipf and M. Welling. "Semi-Supervised Classification with Graph Convolutional Networks." ICLR 2017.
- W. L. Hamilton, R. Ying, and J. Leskovec. "Inductive Representation Learning on Large Graphs." NeurIPS 2017.
- K. Xu, W. Hu, J. Leskovec, and S. Jegelka. "How Powerful are Graph Neural Networks?" ICLR 2019.
- F. Wu, T. Zhang, A. H. de Souza Jr., C. Fifty, T. Yu, and K. Q. Weinberger. "Simplifying Graph Convolutional Networks." ICML 2019.
- U. Alon and E. Yahav. "On the Bottleneck of Graph Neural Networks and its Practical Implications." ICLR 2021.

Notes

The `rough/` directory contains earlier, exploratory notebooks (`Basics.ipynb`, `DeepWalk.ipynb`) written during initial development, along with a duplicate copy of the final notebook and a previously generated PDF export. These are retained for development history and are not required to reproduce the results.

`GCN_and_GraphSAGE/data/` is the Cora dataset cache that PyTorch Geometric writes on first run; it is excluded from version control (see `.gitignore`) since it is reproduced automatically rather than needing to be tracked.